

NETELIXIR · AIGNITION 3.0

ROAScast

Probabilistic Revenue & ROAS Forecasting for E-commerce Ad Spend — across Google, Microsoft/Bing & Meta Ads.

"Forecast accuracy comes from the trained forecasting model; the LLM only explains the forecast — with guardrails."

LightGBM Quantile Model

P10 / P50 / P90 Ranges

100% Offline Scoring

Calibrated Intervals ~92%

Pinned & Reproducible

FastAPI + React + Guardrailed LLM

Project Submission Document

Track 1 scored core + product layer

Team Kryptonite

Avinash Gehi · Hardik Hinduja · Sahil Deshmukh
VES Institute of Technology (VESIT)

Contents

01	Executive Summary	08	Output Schema
02	Problem & Approach	09	Commands & Runbook
03	System Architecture	10	Validation & Results
04	Repository Structure	11	AI Media Planner & Guardrails
05	Models Used (in depth)	12	Reproducibility
06	Feature Engineering	13	Assumptions & Limitations
07	Data Ingestion & Official Dataset	14	Tech Stack & Team

The one-command grading contract. Everything the judge needs runs offline, loads a committed pre-trained model, and writes a single predictions.csv :

```
./run.sh ./data ./pickle/model.pkl ./output/predictions.csv
```

Works with no arguments (same defaults) and via `bash run.sh ...` . Verify the whole contract in one shot with `bash scripts/full_submission_test.sh` .

01 Executive Summary

ROAScast predicts **probabilistic e-commerce revenue and ROAS** from Google, Microsoft/Bing and Meta ad exports — at the **blended / channel / campaign-type / campaign** levels over **30 / 60 / 90-day** windows.

~92%

P10–P90 COVERAGE

~72%

WAPE ON P50

372

PREDICTION ROWS

25,562

ROWS TRAINED ON

109

CAMPAIGNS

WHAT IT DELIVERS

A reproducible, **offline forecasting core** that runs end-to-end with no network access, loads a committed pre-trained model, and writes one `predictions.csv` with calibrated P10/P50/P90 revenue and ROAS ranges for every level and window.

WHY IT IS TRUSTWORTHY

Prediction and reasoning are **separated**. The trained model produces every number; the LLM only explains those numbers under strict guardrails and can never invent forecasts. A **network wall** keeps the scored path fully deterministic.

HEADLINE DIFFERENTIATORS

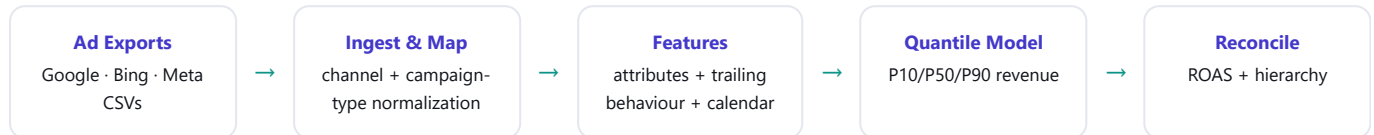
- **Probabilistic, not point.** Three quantile regressors give honest P10/P50/P90 ranges — decision-makers see downside and upside, not a single fragile number.
- **Calibrated intervals.** A time-based split learns one interval-width multiplier so out-of-sample coverage tracks its target (~80–92%) — the point forecast is never distorted.
- **Coherent hierarchy.** Campaign predictions aggregate upward: campaign → campaign-type → channel → blended, always summing consistently.
- **Out-of-distribution scale guardrail.** On data far outside the training scale the system lowers confidence and blends toward a robust ROAS baseline instead of over-trusting the model.

02 Problem & Approach

Advertisers need to know, per campaign and channel, **how much revenue a given ad budget will return** over the next 30/60/90 days — and how confident that estimate is. Historical campaign data is **short, multi-series and sparse**, and the held-out test set contains **brand-new campaigns** never seen in training.

The core design decision

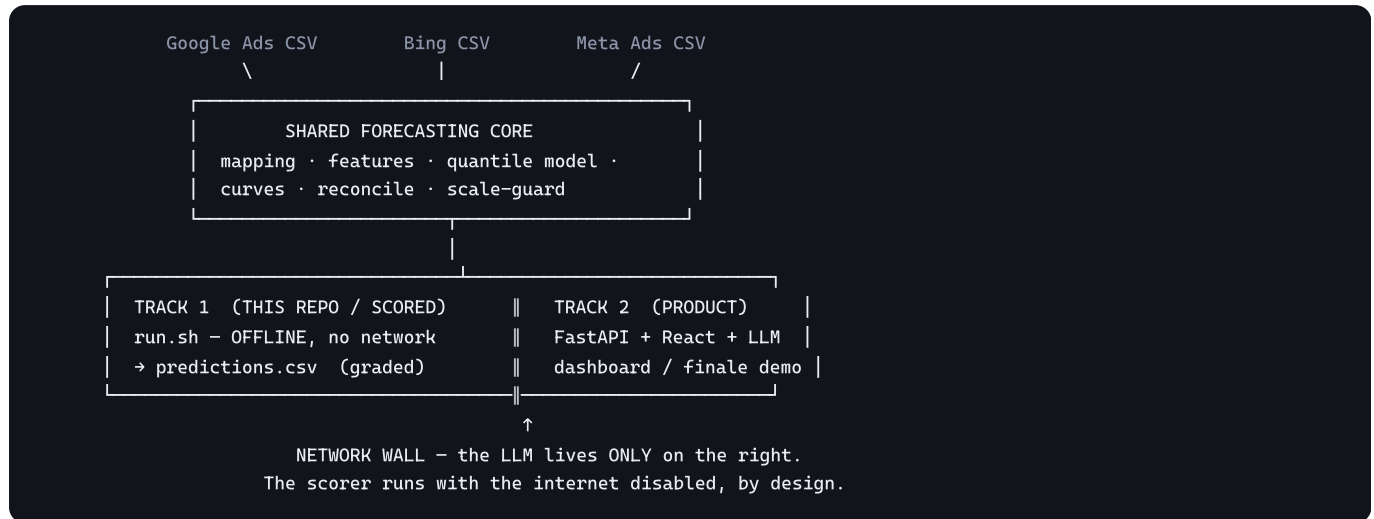
Rather than fitting one time-series model per campaign (ARIMA / Prophet), ROAScast trains a **single global gradient-boosting model** that pools across all campaigns and lets **budget enter as a feature**. This generalizes far better on short, sparse data and — crucially — can score a brand-new campaign from its **attributes and behaviour** alone, with no campaign-identity features to overfit.



Spend is a known input, not a forecast target. ROAScast forecasts *revenue given a budget* and derives **ROAS = revenue ÷ spend**. ROAS is never modelled directly — this keeps the model honest and the what-if simulator meaningful.

03 System Architecture

One repository, two halves, separated by a strict **network wall**. Both halves import a single shared forecasting core, so the live demo and the scored output can never disagree on the math.



TRACK 1 — SCORED CORE (OFFLINE)

- `run.sh` only generates features and predicts — **never** retrains or installs the product layer.
- No network calls, no LLM, no API keys.
- Loads the committed `pickle/model.pkl` and writes one CSV.
- Minimal, pinned dependencies (`requirements.txt`).

TRACK 2 — PRODUCT LAYER (DEMO)

- **FastAPI** backend imports the same core (`/upload /validate /forecast /simulate /explain`).
- **React + Vite** dashboard: Upload, Validation, Forecast, Breakdown, Simulator, Insights.
- Guardrailed LLM narratives on `/explain` only.
- Deps live in `requirements-product.txt` — `run.sh` never installs them.

Why a shared core matters. Because both tracks call the identical `src/forecasting/` package, a number shown in the demo is the same number the grader scores. There is no "demo model" vs "real model" drift.

04 Repository Structure

The layout matches the submission contract exactly — `run.sh` at the root, a committed model under `pickle/`, sample data under `data/`, and the shared core under `src/forecasting/`.

```
roastcast/
├─ run.sh                # entry point - exact name, repo root, executable
├─ requirements.txt      # MINIMAL + pinned (scored core only)
├─ requirements-product.txt # product deps - run.sh NEVER installs these
├─ README.md · COMMANDS.md # methodology + full command runbook
├─
├─ data/                # committed sample CSVs; overwritten at test time
│  └─ google_ads_campaign_stats.csv
│  └─ bing_campaign_stats.csv
│  └─ meta_ads_campaign_stats.csv
├─ pickle/
│  └─ model.pkl          # committed trained model; loads offline (~3 MB)
│  └─ training_profile.json # training-scale metadata for the OOD guardrail
├─ src/
│  └─ generate_features.py # data/ → _features.parquet (CLI)
│  └─ predict.py          # features + model → predictions.csv (CLI)
│  └─ train.py            # OFFLINE; writes model.pkl (NOT in run path)
│  └─ backtest.py         # time-based validation report
│  └─ report.py           # visual range-bar HTML report
│  └─ make_sample_data.py # synthetic sample-data generator
│  └─ forecasting/       # THE SHARED CORE (imported by predict AND the API)
│     └─ schema.py        # output columns + currency - single source of truth
│     └─ ingest.py        # CSV discovery + hard validation
│     └─ mapping.py       # Google/Meta/MS channel + campaign-type normalization
│     └─ features.py      # feature engineering (no campaign identity)
│     └─ model.py         # ForecastModel: train / load / predict
│     └─ curves.py        # per-channel budget→revenue Hill curves
│     └─ reconcile.py     # bottom-up hierarchy aggregation
│     └─ scale_guard.py    # out-of-distribution scale guardrail
├─ product/             # TRACK 2 - behind the network wall
│  └─ backend/app/       # FastAPI: main · core · llm · models · sessions
│  └─ frontend/src/      # React + Vite: 6 dashboard pages
├─ scripts/
│  └─ full_submission_test.sh # end-to-end grading self-test
│  └─ validate_submission.py  # 14-check output validator
│  └─ inspect_data_parsing.py # per-file parse sanity check
├─ reports/             # generated QA + accuracy write-ups
├─ tests/               # pipeline · scale-guard · LLM-guardrail tests
└─ output/predictions.csv # the graded artifact
```

Single source of truth. The exact scored schema (column names, granularities, one-file-vs-per-window) lives in **one place** — `src/forecasting/schema.py` — so matching any announced format change is a one-line edit.

05 Models Used — in depth

ROAScast is an **ensemble of a machine-learning model and a robust statistical baseline**, wrapped in a calibration layer, a saturating-curve layer, and an out-of-distribution guard. Every model below is fitted offline and travels inside `pickle/model.pkl`.

Model A · LightGBM quantile regressors core

Property	Detail
Algorithm	Gradient-boosted decision trees — LightGBM 4.6.0 , one regressor per quantile
Objective	<code>quantile</code> loss at $\alpha = 0.1 / 0.5 / 0.9 \rightarrow P10 / P50 / P90$ directly
Target transform	Fitted on <code>log1p(revenue)</code> , inverted with <code>expm1</code> — skew-robust; quantiles are invariant under monotone transforms so P10/P50/P90 stay valid
Key hyper-params	<code>n_estimators=400 · num_leaves=31 · learning_rate=0.05 · min_child_samples=20 · subsample=0.9 · colsample_bytree=0.9 · reg_lambda=1.0</code>
Determinism	<code>deterministic=True · force_row_wise=True · random_state=42 · n_jobs=1</code> — bit-reproducible
Quantile-crossing fix	The three predictions are sorted row-wise so <code>P10 ≤ P50 ≤ P90</code> is always guaranteed
Portability	Stored as LightGBM model strings (not live Booster objects) — robust across library patch versions, the #1 cause of cold-grader load failures

Model B · Seasonal-ROAS baseline robust

A purely arithmetic, extremely robust estimate: **trailing 28-day ROAS × planned spend × seasonal index**. It anchors the ensemble when the ML model is uncertain and is the fallback the OOD guard blends toward.

Ensemble P50 blend

The point forecast is `P50 = w · (LightGBM) + (1 - w) · (seasonal-ROAS baseline)`. The blend weight **w** is chosen by **WAPE on a forward time-based calibration split** from a LightGBM-heavy grid (1.0, 0.85, 0.7, 0.5); ties favour more GBM and thin data defaults to pure GBM. Weights are learned **per forecast window** (30/60/90) when there is enough calibration data, else pooled.

Calibrated prediction intervals differentiator

The same time-based split (earliest 80% fit / latest 20% calibrate) learns a single **interval-width multiplier f** via binary search so out-of-sample coverage approaches the ~80% target. Coverage is monotone in f, making the search stable. **The point forecast P50 is never altered** — only the distance of P10 and P90 from it. Backtested coverage lands at ~92% (conservative side).

Budget-response curves (per channel) simulator

A **saturating Hill curve** is fitted on historical `(spend, revenue)` pairs per channel in pure NumPy (no scipy). It makes diminishing returns explicit, powers the what-if simulator, and is **monotone increasing by construction** — guaranteeing "revenue cannot fall as budget rises." Curves are independent per channel (deliberately not a media-mix model).

Monotonicity note. LightGBM rejects `monotone_constraints` under the quantile objective, so the budget→revenue guarantee is enforced **structurally**: (a) response curves are monotone by construction, and (b) any budget sweep through the GBM passes through an isotonic cumulative-max step (`predict_budget_sweep`).

Reconciliation

Predictions are made at the **campaign** level and aggregated **upward** to campaign-type → channel → blended. Summing medians is exact; summing P10/P90 bounds is a stated additive approximation. ROAS at each level = revenue ÷ total planned spend at that level.

Out-of-distribution scale guardrail

Training-scale metadata (spend/revenue quantiles, ROAS range, channels, campaign types, date range) is saved to `pickle/training_profile.json`. At predict time uploaded data is compared against it to produce an **ood_score ∈ [0,1]** and a High/Medium/Low confidence bucket:

Confidence	Behaviour
High	Trained model used unchanged — official-scale output is bit-identical
Medium	Blend revenue quantiles 60/40 with the trailing-ROAS baseline; widen P10–P90 by the OOD score
Low	Blend 25/75 toward the baseline; widen intervals further. ROAS always re-derived as revenue ÷ spend

Model C · The LLM (product layer only)

The LLM is **never** part of scoring. It only explains forecast outputs under guardrails — see [Section 11](#). The forecasting model predicts; the LLM explains; guardrails verify.

06 Feature Engineering

Features are computed per **campaign × window**. The guiding rule: **never use campaign identity** — the held-out set has unseen campaigns, so identity features would assign garbage. Only attributes and behaviour are used.

ATTRIBUTES

channel · campaign_type · is_brand

BUDGET (KNOWN INPUT)

`budget_input` — planned spend for the window. Derivation isolated in one place (`features.derive_budget_input()`).

TRAILING BEHAVIOUR (14 / 28-DAY)

- Revenue, spend, ROAS, conversions, clicks, impressions
- Derived rates: CTR, CVR, CPC
- Momentum: week-over-week / month-over-month growth, daily spend rate, active days

CALENDAR & SEASONALITY

- Month, week-of-month, trend index, a learned monthly seasonal index
- An explicit **US retail calendar** — Black Friday, Cyber Monday, the Nov–Dec peak. A 30/60/90-day window crossing BFCM moves revenue enormously, so the model must know.

Defensive computation. Every feature falls back gracefully: group mean → channel mean → global mean → 0. **No NaN ever reaches the model**, and no entity the grader expects is ever silently dropped.

07 Data Ingestion & Official Dataset

`data/` is parsed **dynamically** — every CSV is mapped by filename channel inference first, then column aliases, then column-signature detection. The three official Algnition files are natively supported.

File	Channel	Date	Spend	Revenue	Conversions
google_ads_campaign_stats.csv	google	segments_date	cost_micros ÷ 1e6	conversions_value	conversions
bing_campaign_stats.csv	microsoft	TimePeriod	Spend	Revenue	Conversions
meta_ads_campaign_stats.csv	meta	date_start	spend	conversion → revenue	none → 0

Hard validation (fails loudly)

`src/forecasting/ingest.py` aborts **before** writing output when: an official file parses to 0 rows; > 5% of rows land in channel `other` ; total revenue is 0 while revenue columns exist; or a per-row spend suggests `cost_micros` was not divided. Unknown columns never crash the run; rows with invalid dates are dropped; all metrics coerce to non-negative numerics.



Dataset span: 2024-01-01 → 2026-06-05 · 25,562 total rows · 109 campaigns across three platforms.

08

Output Schema — predictions.csv

One file covers all three windows. Columns, in order:

Column	Meaning
level	blended · channel · campaign_type · campaign (coarser levels leave finer columns blank)
channel	google · meta · microsoft · other
campaign_type	Search · Shopping · PerformanceMax · Display · Video · DemandGen · Prospecting · Retargeting · Brand · DPA · Other
campaign	campaign name (campaign-level rows only)
window_days	30 · 60 · 90 — aggregate total for the window (never daily)
revenue_p10/p50/p90	revenue quantiles, USD
roas_p10/p50/p90	ROAS quantiles — a dimensionless multiple (never \$)

Guarantees. $P_{10} \leq P_{50} \leq P_{90}$ holds for both revenue and ROAS, and the hierarchy is **coherent**: campaign sums to campaign-type sums to channel sums to blended. Output is **372 rows** = all four levels × windows 30/60/90.

09 Commands & Runbook

Copy-paste blocks. The scoring core is fully offline (no keys, no internet, no LLM). The product layer is optional and demo-only. On Windows use Git Bash.

① The judge command — most important

```
# one-time: create venv + install the PINNED scoring deps
python -m venv .venv
source .venv/Scripts/activate          # macOS/Linux: source .venv/bin/activate
python -m pip install --upgrade pip
python -m pip install -r requirements.txt

# THE JUDGE COMMAND - generates features + predictions in one shot
./run.sh ./data ./pickle/model.pkl ./output/predictions.csv

# also valid:
./run.sh                               # uses the same defaults
bash run.sh ./data ./pickle/model.pkl ./tmp/custom.csv # custom output path
```

② Validate the output — must PASS before submitting

```
python scripts/validate_submission.py ./output/predictions.csv
# 14 checks: schema, no NaN/inf/negatives, windows 30/60/90, all 4 levels,
# P10 ≤ P50 ≤ P90, official channels present, hierarchy coherent. Exit 0 = PASS.
```

③ Full submission self-test — before final submission

```
bash scripts/full_submission_test.sh
# model load, explicit/default/custom/dynamic-dir/no-key runs, 5x stability,
# freshness (no-append), validating after every run.
# Prints "FINAL SUBMISSION TEST PASS" at the end.
```

④ Inspect data parsing & ⑤ Accuracy backtest

```
python scripts/inspect_data_parsing.py --data-dir ./data # per-file raw vs parsed rows
python src/backtest.py --data-dir ./data                 # WAPE, MAPE, pinball, coverage
```

⑥ Unit + guardrail tests

```
python -m pip install pytest
python -m pytest tests/ -q    # 15 tests: pipeline end-to-end + AI guardrails. All pass.
```

⑦–⑧ Optional product layer (demo only)

```
# Backend - FastAPI
cd product/backend
python -m pip install fastapi "uvicorn[standard]" pydantic python-multipart
uvicorn app.main:app --reload --port 8000
# Endpoints: GET /health · POST /upload /validate /forecast /simulate /explain

# Frontend - React + Vite (new terminal)
cd product/frontend
npm install
npm run dev    # proxies /api → :8000; falls back to mock data if backend is down
npm run build  # production build into dist/
```

Regenerate everything (offline dev)

```
python src/train.py --data-dir ./data --out ./pickle/model.pkl # retrain
python src/backtest.py --data-dir ./data # validation report
python src/report.py --predictions ./output/predictions.csv --out ./output/report.html
python -m pytest tests/ -q
```

10 Validation & Results

A **time-based backtest** mimics the scorer: train on an earlier period, hold out the forward 30/60/90-day window, predict, and score against realized actuals — on the official Google + Bing + Meta dataset (25,562 rows, 109 campaigns, 2024-01-01 → 2026-06-05).

Metric	Result	Notes
Interval coverage (P10–P90)	~92%	fraction of actuals inside the band (target ~80%; conservative side)
WAPE on P50	~72%	volume-weighted campaign-level revenue error over the 30/60/90d holdout
Pinball P10 / P50 / P90	1,664 / 7,097 / 9,436	the standard probabilistic scoring metric
Best window	30 days	reported per window and per channel
Best channel	meta	coverage/WAPE/MAPE broken out by channel

Honest reporting. MAPE is reported but dominated by near-zero-revenue campaigns, so **WAPE is the honest aggregate number**. The backtest also reports per-window and per-channel coverage, parsed spend/revenue totals, campaign counts and the date range. Full write-up in `reports/backtest_report.md`.

Quality assurance

- **15 automated tests** — pipeline end-to-end, output contract & coherence, scale-guard, and LLM no-hallucination guardrails. All pass.
- **14-point output validator** — schema, no NaN/inf/negatives, all windows and levels present, monotone quantiles, coherent hierarchy.
- **Full submission self-test** — runs the grader command repeatedly for stability and freshness, validating after every run.

11 AI Media Planner & Guardrails

The ML model predicts · the LLM explains · guardrails verify. The AI layer is product-only and never touches `run.sh`. The forecasting model is trained on the campaign data; the LLM is **not** — it is grounded at runtime with structured forecast outputs and can never generate forecast numbers.

Provider chain — first available wins



No key is required. With zero keys and no Ollama the AI Insights page still works via the deterministic rule-based fallback. Any one key in `product/backend/.env` unlocks a live narrative — and can never change forecasts or scored output.

Guardrails (grounding)

- The LLM receives only a compact JSON context with explicit `allowed_channels` / `allowed_campaigns` / `allowed_metrics`; **temperature 0**; JSON-only responses.
- A post-validator **rejects**: invented numbers, unknown campaign/channel names, missing evidence references, deterministic language ("will definitely", "guaranteed"), and unsupported causality ("because of competitors / inflation / algorithm changes").
- Rejected responses fall back to the rule-based generator; the UI shows `Guardrail: Passed` or `Fallback used`.
- Covered by `tests/test_llm_guardrails.py` — invented numbers, fake campaigns, unsupported causality, provider-chain failure, keyless fallback.

Provider	Env variable	Default model
Ollama (local, no key)	OLLAMA_HOST	qwen3:8b
OpenAI	OPENAI_API_KEY	gpt-4o-mini
Google Gemini	GEMINI_API_KEY	gemini-2.0-flash
Groq	GROQ_API_KEY	llama-3.3-70b-versatile

12 Reproducibility

- **Pinned dependencies** — only installed & tested versions in `requirements.txt`.
- **All randomness seeded** — NumPy & LightGBM with `deterministic=True`.
- **No absolute paths** — relative paths / passed arguments only.
- **Model committed** under `pickle/` (~3 MB, no Git LFS).
- Stores LightGBM **model strings**, not Booster objects — portable across patch versions; verified to unpickle in a clean virtualenv.
- `data/` read **dynamically** (glob + column-signature detection) — assumes the schema, not the row count, and tolerates unseen campaigns.

PINNED SCORING STACK

numpy	2.2.6
pandas	2.2.3
scikit-learn	1.6.1
lightgbm	4.6.0
pyarrow	18.1.0
joblib	1.4.2
scipy	1.15.3

Python 3.13 (developed & tested); wheels resolve on CPython 3.10–3.13.

13 Assumptions & Limitations

ASSUMPTIONS

- **Spend is a known input**, not a forecast target — set in one place (`features.derive_budget_input`).
- **Existing platform attribution is the source of truth**, used as-is; no custom attribution engine.
- **Blended total = simple sum** across Google + Microsoft + Meta; cross-platform overlap is acknowledged, not deduplicated.
- All monetary values are **USD**, native to the exports (no FX conversion).

LIMITATIONS

- **Limited history** → **coarse seasonality** — ~14 months sees one BFCM peak; more history sharpens it.
- **Interval-summing approximation** — aggregated P10/P90 bounds are summed (medians remain exact).
- **Per-channel curves, not an MMM** — cross-channel halo/cannibalization intentionally not modelled.
- **Meta conversion counts unavailable** in the official export — Meta CVR features fall back to safe defaults.

14

Technology Stack & Team

Scored core

Python 3.13

LightGBM

NumPy

pandas

PyArrow

scikit-learn

scipy

Product backend

FastAPI

Uvicorn

Pydantic

Product frontend

React

Vite

Tailwind CSS

AI layer (optional)

Ollama

OpenAI

Gemini

Groq

Deterministic fallback

TEAM KRYPTONITE

Project: ROAScast
College: Vivekanand Education Society's Institute of Technology (VESIT)
Event: NetElixir Algnition 3.0

Member
Avinash Gehi
Hardik Hinduja
Sahil Deshmukh @sahil200511

ROAScast — the trained forecasting model produces calibrated P10/P50/P90 revenue & ROAS ranges; the AI Media Planner only explains them, under strict guardrails. Prediction and reasoning, cleanly separated.

Built for NetElixir Algnition 3.0 · Track 1 scored core + product layer · Offline · Reproducible · Guardrailed.